

Partition Placement and Resource Allocation for Multiple DNN-Based Applications in Heterogeneous IoT Environments

Taeyoung Kim, Hyungbin Park[✉], Younghwan Jin[✉], Seung-Seob Lee[✉], and Sukyoung Lee[✉]

Abstract—The evolution of the Internet of Things (IoT) has been driving the explosive growth of deep neural network (DNN)-based applications and processing demands. Hence, edge computing has emerged as a potential solution to meet these processing requirements. However, emerging IoT applications have increasingly demanded to run multiple DNNs to extract multifaceted knowledge, requiring more computational resources and increasing response time. Consequently, edge nodes cannot act as a complete substitute for the previous cloud paradigm, owing to their relatively limited resources. To address this problem, we propose to incorporate nearby IoT devices when allocating resources to multiple DNN models. Furthermore, the optimization of resource allocation can be hindered by the heterogeneity of IoT devices, which affects the delay performance of DNN-based computing. In this context, we propose a DNN partition placement and resource allocation strategy that considers different processing powers, memory, and battery levels for heterogeneous IoT devices. We evaluate the performance of the proposed strategy through extensive simulations. Simulation results reveal that the proposed strategy outperforms other existing solutions in terms of end-to-end delay, service probability, and energy consumption. The proposed solution was further simulated in a Kubernetes testbed consisting of actual devices to assess its feasibility.

Index Terms—Heterogeneous Internet of Things (IoT), multiple deep neural networks (DNNs), partition placement, resource allocation, utility.

I. INTRODUCTION

THE Internet of Things (IoT) has become more prevalent in businesses and daily lives than ever, providing vast opportunities for many novel services in various domains, such as smart cities, healthcare, automotive, and smart factories, by analyzing data sensed from the environment. However, with the explosion in big data, there has been a paradigm shift from centralized cloud computing to edge computing as a response to the massive amount of data sent to the cloud, which causes a substantial delay that results in the failure to

meet the response time requirements of IoT applications. In particular, the rapid development of embedded deep neural network (DNN) functions in smart end devices has expanded AI capabilities from the remote cloud to the IoT device, and enabling the remote cloud to perform DNN inference with multiple IoT devices in a collaborative manner [1], [2], [3]. Nevertheless, edge computing cannot be a complete substitute for the cloud computing paradigm because the computing and storage capacities of edge nodes are limited compared to that of traditional clouds [3], [4], [5], [6], [7].

As edge computing has been widely considered beneficial to IoT devices, many previous studies have tried to address the problems in IoT edge systems, such as resource allocation and computation offloading [6], [7], [8], [9], [10], [11]. Among them, considering DNN-based applications, collaborative deep learning strategies have been recently proposed in [8], [9], [10], and [11]. However, most of these studies did not consider the available resources of nearby IoT devices, focusing only on edge nodes in preconfigured locations.

Moreover, in the collaborative execution of DNN functions with nearby IoT devices, the optimization of resource allocation can be hindered by the heterogeneous and variable capabilities of IoT devices, thereby affecting the performance of DNN-based computing [10], [11]. In other words, the heterogeneity of hardware specialization has inconsistent effects on the response times for DNN-based applications. To address this “heterogeneity” issue, it is critical to consider the heterogeneous performance of IoT devices available for supporting DNN-based applications. Although previous studies have considered heterogeneity, they are either specialized for specific DNN tasks, such as video analytics or convolutional neural networks (CNNs) [10], [11], [12], [13] or lack consideration for the distributed deployment of partitioned DNN models [14], [15], [16], [17]. This motivates us to incorporate the diverse characteristics of IoT devices when placing partitioned DNN models in the IoT system.

Meanwhile, the ever-growing IoT applications have increasingly demanded the simultaneous execution of multiple DNN models, further exacerbating the aforementioned insufficiency of computing resources. For instance, while images of surroundings are captured, multiple different types of DNN models, such as YOLO [18] and FaceNet [19], need to be executed to infer different information of faces and objects from the same image, which increases the request for computational

Manuscript received 31 July 2022; revised 20 December 2022; accepted 3 January 2023. Date of publication 11 January 2023; date of current version 23 May 2023. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea Government (MSIT) under Grant 2022R1A2B5B01001683. (Corresponding author: Sukyoung Lee.)

Taeyoung Kim, Hyungbin Park, Younghwan Jin, and Sukyoung Lee are with the Department of Computer Science, Yonsei University, Seoul 03722, South Korea (e-mail: sklee@yonsei.ac.kr).

Seung-Seob Lee is with the Department of Computer Science, Yale University, New Haven, CT 06511 USA.

Digital Object Identifier 10.1109/JIOT.2023.3235993

resources and response time [9], [20]. However, existing studies on DNN-based applications lack support for the simultaneous execution of DNN models [10], [11]. Although multiple DNN models were considered in [9], it only utilizes the edge server's resource and does not take advantage of the IoT devices' resource. Furthermore, in [9], dependencies in DNN models were randomly given, regardless of the type of DNN model, which could potentially blur the characteristics of each model. This motivates us to further consider multiple DNN models with different dependency structures when developing our resource allocation strategy. Additionally, there have been concerns regarding the energy efficiency of these DNN models during their deployment as 6G, and other future networks are on the rise and workload is offloaded via networks [21], [22]. We further investigate the waiting delay in an edge node and IoT devices when multiple DNN models are to be executed, reflecting the case where there is a surge in the number of requests for IoT services [6], [8], [23].

To address the aforementioned issues, we propose a distributed DNN computing framework that provides multiple DNN-based applications with optimal latency in an IoT system consisting of heterogeneous devices. The main contributions of this study are summarized as follows.

- 1) We propose a DNN partition placement and resource allocation (PRA) scheme that coordinates collaborative inference over IoT devices in proximity. As IoT devices are heterogeneous, with a wide spectrum of hardware capabilities and operating systems, we base our PRA scheme on containerization to address the aforementioned heterogeneity issue. The proposed PRA scheme offers *hardware-independent placement of DNN partitions on IoT devices* by placing each DNN partition as a service running inside a container on the edge server or IoT device.
- 2) We develop a problem formulation that places the containerized partitions of multiple DNN models and allocates computing and memory resources to partitions in IoT devices as well as the edge node with the aim to maximize utility while serving the multiple DNN-based applications. We model the utility in terms of end-to-end (E2E) delay, service probability for multiple DNN models, and energy consumption of IoT devices. In the problem formulation, *we numerically incorporate different computing powers, memory capacities, and battery levels of individual IoT devices*, so that the proposed PRA can cope with the heterogeneity of IoT devices, ultimately solving the heterogeneity issue.
- 3) When multiple DNNs applications are requested at the same time, the edge node can be easily overloaded, leading to an unexpected performance bottleneck. *For a more accurate estimation of E2E delay for multiple DNN applications, we analyze the waiting time at both edge nodes and IoT devices.* The M/D/1/K queueing system is adopted to investigate the service probability indicating how many requests can be processed among the requested multiple DNN models, in addition to the analysis of waiting time.

- 4) We leverage deep reinforcement learning (DRL) based on the soft actor-critic (SAC) method to provide an efficient algorithm for solving the problem formulation because DRL can help the proposed algorithm adjust the resource allocation for multiple DNN partitions to achieve the optimal long-term reward. The DRL learns the dynamic features of IoT devices and requests for DNN-based applications and utilizes the features learned (i.e., available computing and memory resources, battery level, and the number of requests) to allocate resources for multiple DNN partitions.
- 5) Extensive simulations verify the effectiveness of the proposed DNN partition PRA strategy. The simulation results demonstrate that the proposed strategy performs better than the existing solutions in terms of utility. Furthermore, *we implemented and experimented our PRA scheme in a testbed with different IoT devices and DNN models by using Docker and Kubernetes* to demonstrate that our container-based PRA scheme performs well in a real heterogeneous IoT environment.

The remainder of this article is organized as follows. In Section II, we discuss related works regarding DNN PRA. Then, we discuss our IoT system and the system architecture in Section III. Section IV formulates the problem based on the model of the IoT environment and DNN partitions. In Section V, we propose a solution for the complex problem of distributed computing in an IoT environment in Section V. Section VI evaluates the performance of the proposed algorithm and discusses the results of our testbed simulation. Finally, we draw the conclusion of our work in Section VII.

II. RELATED WORK

A. Distributed DNN Placement and Offloading

Machine learning (ML) technologies are used to provide intelligent services for IoT systems, including behavioral profiling, monitoring, prediction, planning, and perception [3], [4]. Previous studies have discussed IoT-edge offloading methods for DNN computations to provide intelligent IoT services relying on DNNs. Zhang et al. [7] proposed an adaptive method for offloading visual data to the edge for computation. Lin et al. [24] utilized edge-based reinforcement learning (RL) for determining the offloading decision and resource allocation for coordination between the local device and the edge. To further increase the efficiency of DNN computation, Xu et al. [25], Shi et al. [26], Li et al. [27], Chen et al. [28], and Zeng et al. [29] proposed offloading a partition of a DNN model to the edge and utilizing on-device computation. Zeng et al. [29] also discussed the tradeoff between accuracy and model pruning as well as determining an offloading point between an edge and local device for reducing inference delay. Tang et al. [30] offers a layerwise partition offloading scheme that adapts the offloaded partition according to the local device's capability in a multiuser edge offloading setting to minimize latency. However, both works only discuss bipartitioning and do not discuss multipartitioning nor partition placement on multiple devices.

To further expand computational resources, studies have been conducted on distributed inference through offloading between multiple devices. Zhou et al. [10] and Zeng et al. [11] proposed to partition the DNN inference workload based on computing capabilities and network conditions of nearby devices. However, these studies assumed that computation and communication resources in these devices are static and known for their partitioning. With this assumption of a fixed device set, advantages of edge computing are limited in terms of the edge's proximity to the user in need, as addressed in [31] and [32]. To overcome this, Sami and Mourad [31], [32] proposed a framework capable of forming fog devices on the fly with any volunteering device and then extended their work to the context of vehicular fog computing [33]. However, their work optimized the number of services pushed to IoT devices, assuming that the resources required by each service are static and do not deal with DNN-based applications. Moreover, these solutions lack support for multiple DNN models that must be executed simultaneously. He et al. [9] extended the partition deployment problem by considering multiple DNN models and proposed a heuristic algorithm based on Markov approximation for mainly tackling the DNN partition deployment and solving resource allocation of partition as a subproblem across edge servers, achieving state-of-the-art performance. However, they did not consider the resources of nearby IoT devices. Additionally, different dependency structures of multiple DNN models were not addressed in [9], which can critically impact the DNN execution time when configuring partition placement.

B. Consideration of Heterogeneous IoT Devices

Because IoT devices have heterogeneous computing capabilities, aimed at different IoT tasks varying from reading temperature to video analytics, multiple studies have considered the heterogeneity of IoT devices in their resource allocation schemes [34]. LAVEA [12] proposed an offloading scheme for low-latency video analysis services considering the computing resources of IoT devices, edge nodes, and the remote cloud. In [14] and [15], a container-based IoT service framework was proposed to enable IoT service offloading across heterogeneous devices, with extensive evaluations of the computing capability of heterogeneous IoT devices. Boovaraghavan et al. [16] proposed an ML framework to automate the E2E life cycle of ML, from learning to serving ML models. Jin et al. [17] proposed a reliable distributed deep learning framework with self-awareness of the computing capabilities of edge clusters consisting of IoT devices. Although previous studies have considered the heterogeneity of IoT devices, only [12] studied the resource allocation for distributed DNN models. Moreover, Yi et al. [12] only focused on a specific task (i.e., video analytics).

C. Deep Learning-Based Resource Allocation for the Edge

Recently, deep learning-based methods have been studied to determine resource allocation in heterogeneous edge computing and cloud computing environments in a robust manner. To meet the delay requirements of mobile users with the assistance of mobile cloud computing and the mobile edge

computing (MEC) environment, Wu et al. [35] proposed a distributed deep learning-based resource allocation algorithm that considers task completion delay and energy consumption. To accommodate the changing edge environment and preference for applications in user devices, Chen et al. [8] proposed a multiple buffer deep deterministic policy gradient (MB-DDPG) for the resource allocation of multiple microservice placements, optimizing for minimum waiting delay at the devices. However, Chen et al. [8] did not consider containerization in IoT devices while Wu et al. [35] did not consider containerization at all. Wang et al. [36] proposed a DRL-based resource allocation scheme to overcome the performance limitations imposed by the changing network conditions in the MEC environment. Chen et al. [8], Wu et al. [35], and Wang et al. [36] did not consider distributed DNN placement.

D. Containerized Services

IoT has evolved into a huge ecosystem consisting of multiple servers, applications, sensors, and protocols. Various IoT components must be integrated with each other, leading to an increase in deployment time and cost. Thus, containerization has also become the de-facto approach because containers provide operating system-level virtualization in heterogeneous IoT environments, enabling the placement of on-demand services and dynamic reconfiguration of device resources. Furthermore, containers can run on IoT devices characterized by limited computing resources because their lightweight architecture requires less resources and lower bootstrapping time than virtual machines [32]. Recent studies have examined the deployment of containerized services at the IoT edge. Chen et al. [8] utilized containers for microservices running on IoT devices. To solve the problem of nonuniformity among resources of volunteering devices, Sami et al. proposed allocating containerized resources of devices with evolutionary memetic algorithms [32], [33]. Jin et al. [17] further discussed the optimizations made through the containerization of DNN services on IoT devices. Previous studies [32], [33] do not consider deep learning methods for solving the resource allocation problem when deploying containerized services. Chen et al. [8] utilized the MB-DDPG deep learning method to adapt resource allocation to a dynamic environment, but it did not consider the deployment of DNN service nor its characteristics.

In Table I, we summarize the differences between our PRA algorithm and the main related works.

III. SYSTEM ARCHITECTURE

A. IoT Environment

As shown in Fig. 1, we consider an IoT environment consisting of an edge computing gateway and heterogeneous IoT devices denoted by $D = \{1, \dots, d, \dots, |D|\}$ with the first being the edge node. This implies that the IoT gateway provides the edge computing capabilities within the radio access network. Because the gateway is usually co-located with a WiFi access point (AP) [37], [38], we assume that the IoT devices are connected to the edge node by wireless link.

We assume that edge computing gateway has larger computing and memory capacities than IoT devices. Thus, it can host

TABLE I
SUMMARY OF THE MAIN RELATED WORKS

Related work	DNN Service	Service Placement	Resource Allocation	IoT as Resource	Container -ization
Our work	Multiple	✓	✓	✓	✓
[7], [35], [36]	-	✓	✓	-	-
[8], [12]	-	✓	✓	-	✓
[9]	Multiple	✓	✓	-	-
[10], [11]	Single	✓	-	✓	-
[14]–[16], [31]	-	✓	-	✓	✓
[17]	Single	-	-	✓	✓
[24]	-	✓	✓	-	-
[26], [27], [29]	Single	✓	-	-	-
[28]	Multiple	✓	-	-	-
[30]	Multiple	-	✓	-	✓
[32]	-	✓	-	-	✓
[33]	-	✓	✓	✓	✓

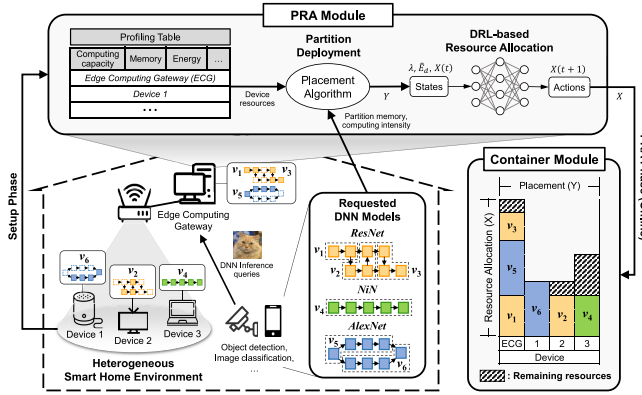


Fig. 1. System architecture of heterogeneous cooperative platform.

more containers than IoT devices to process multiple DNN-based applications. We assume that the available devices are relatively stable for a given time period because many edge environments are maintained statically in private spaces, such as smart homes and smart factories [10], [11], [37]. One use case for our work is the smart home domain where devices, such as speakers, TVs, laptops, and PCs, are leveraged to support DNN-based applications. When assigned, each partition of a DNN is deployed in an IoT device or the edge node in a container.

B. System Components and Overall Workflow

There are several components to this architecture, which include a PRA Module, Profiling Table, Container Module, and Profiler. Container Module is placed inside both every IoT device and edge computing gateway, whereas the PRA Module is only placed in the edge computing gateway. We briefly discuss these components below.

1) PRA Module provides the following functionalities:

- The PRA Module generates a distributed DNN computing strategy using PRA algorithm (detailed in Section V).
- The *Profiling Table* inside the PRA Module contains all device profiles collected by the edge computing gateway.
- This module also has information about the DNN models requested by IoT users (i.e., their number and types).

2) Container Module assists PRA module with the following functionalities:

- Container Module dynamically configures the device's resources to allocate images for the DNN model/partitions.
- The *Profiler* placed in each Container Module creates a profile of the device, which includes computing capacity, memory, energy parameters, and computing intensity of DNN partition [10], [11] and [38].

With these components, the system conducts partition deployment in the following two phases.¹

1) *Setup Phase*: When IoT devices establish connections with a gateway, Container Module in each device downloads container images containing DNN partitions from the cloud/repository. The profiler in each Container Module downloads and runs all the available DNN partitions locally, creates a device profile, and obtains the computing intensity and memory usage of the DNN partition on the device. After creating the profile, each IoT device sends it to the gateway. The gateway stores the profiles in the Profiling Table [11].

2) *PRA Phase*: The PRA phase begins when the Setup phase is completed and consists of the following steps:

- The PRA Module develops a collaborative PRA strategy about where to place the partitions and how much resources to allocate to them by running our proposed PRA algorithm, using the device profiling results collected in the *Setup phase* and the network status, as shown in Fig. 1.
- The DRL-based algorithm performs inference to determine the resource allocation (detailed in Section V-B) along with the deployment decision. The related neural network parameters are pretrained before entering this phase.
- According to the strategy decided by the proposed algorithm, the PRA Module selects the IoT devices and notifies the selected devices of the DNN partitions to be deployed, and the amount of resources that they have to provide for the partitions.
- As instructed by the PRA Module, the Container Module in the selected devices deploys DNN partition images and configures their resources for running the partition images.

Through this phase, the strategy is decided offline. Later, when multiple DNN-based applications are run by IoT users, raising DNN inference queries, the PRA module performs collaborative inference in response to the queries using the placed partition images and IoT resources allocated by the strategy.

IV. PROBLEM FORMULATION

In this section, we first develop our utility model in terms of E2E delay, service probability, and energy consumption. Then,

¹For practical deployment of partitions, the well-known containerization technology, Docker can be utilized along with a container orchestration tool, Kubernetes [32], [39]: Kubeadm is used to form a cluster of a gateway and IoT devices, while Kubectl to deploy the partitions and to conduct the resource allocation. The strategy decided by the PRA algorithm was tested on the Kubernetes platform (Detailed in Section VI-D).

TABLE II
SUMMARY OF MAIN NOTATIONS

Notation	Description
$\mathcal{D}$	Set of IoT devices with the edge node
N	Number of DNN models
V_n	Set of type- n DNN partitions
v_i	i th DNN partition
$v_{\hat{n}}, v_{\bar{n}}$	The first, last partition of type- n DNN model
$\text{pre}(i), \text{suc}(i)$	Set of Preceding, succeeding partitions of v_i
X, Y	Partition deployment, resource allocation strategy
x_i	Allocated computing resource of v_i in strategy X
y_i	Deployment decision of v_i in strategy Y
$S_i^{(in)}, S_i^{(out)}$	Size of input, output data for the partition v_i
$S_i^{(M)}$	Memory capacity required for the partition v_i
C_d, M_d	Computing, Memory capacity of the device d
$\delta_{i,d}$	Computing intensity of the partition v_i per device d .
E_d	Total energy consumption at the device d
T_n	Total delay of type- n DNN model
$U_n(X, Y)$	Utility for type- n DNN model
$I\{\cdot\}$	Indicator function $I\{\cdot\} = 1$ if the condition $\cdot$ holds

we formulate the partition deployment and resource allocation problem with the aim to optimize the utility. For convenience, Table II lists the notations frequently used throughout the rest of this study.

A. Containerized Multiple DNN Models

The partitioning decision for a DNN model is optional, and the partitioning point of a DNN model may change within the set of partitioning points [9]. For a type- n DNN task, all DNN partitions can be processed in containers at IoT devices and/or the edge node. We assume that there are N different DNN models in the environment. We model a type- n DNN as a directed acyclic graph (DAG) $\mathcal{G}_n = (E_n, V_n)$, where E_n and V_n are the sets of edges representing order relations and nodes representing partitions, respectively. Each partition $v_i \in V_n$ is determined by the proposed partitioning strategy, where v_i consists of multiple layers. $\text{pre}(i) = \{p | \forall (v_p, v_i) \in E_n\}$ denotes the set of predecessors for partition v_i and $\text{suc}(i) = \{s | \forall (v_i, v_s) \in E_n\}$ is the set of successors [40]. We utilize p, s to index predecessors and successors. $v_{\hat{n}}$ is the first partition of V_n , and $v_{\bar{n}}$ is the last partition of V_n .

Each partition v_i has the following attributes $x_i, y_i, S_i^{(in)}, S_i^{(out)}$, and $S_i^{(M)}$, which are the computing resource to be allocated, deployment decision, size of input data, size of output data, and memory capacity required for processing the partition v_i . $y_i = d$ indicates that partition v_i is deployed in the IoT device d , and $y_i = 0$ denotes that v_i is not deployed in any device.

B. End-to-End Delay

The E2E delay for tasks requiring DNN in the considered IoT system mainly consists of processing and transmission delay for edge and IoT devices. Let $\delta_{i,d}$ denote the computing intensity (in processing cycles per 1 KB input) of the partition v_i in device d , which is measured by the *profiler* in the Setup phase. Tasks are serviced in partitions at a rate of $\mu_i = (x_i / [S_i^{(in)} \delta_{i,d}])$ if $y_i = d$.

1) *Processing Delay*: Assuming the Poisson arrival of type- n DNN tasks, the average queuing delay can be obtained

using an M/D/1/K queuing model [9], [41], [42], [43]. Mean queuing delay $T_i^{(w)}$ for the containerized partition v_i with arrival rate λ_i of v_i is defined as

$$T_i^{(w)} = \left(K - 1 - \frac{\sum_{k=0}^{K-1} b_k - K}{\rho_i b_{K-1}} \right) \frac{1}{\mu_i} \quad (1)$$

where $\rho_i = (\lambda_i / \mu_i)$

$$b_k = \sum_{j=0}^k \frac{(-1)^j}{j!} (k-j)^j e^{(k-j)\rho_i} \rho_i^j \quad (2)$$

and $b_0 = 1$.

Using $T_i^{(w)}$ and the computing delay $(1/\mu_i)$, we define $T_i^{(pr)}$, the processing delay of v_i , as

$$T_i^{(pr)} = \frac{1}{\mu_i} + T_i^{(w)}. \quad (3)$$

To serve each type- n DNN task, all containerized partitions $v_i (\in V_n)$ must be processed. Let us say that the processing of v_p (i.e., the predecessor of v_i) is completed at t_p and $t_p + \Delta$ for two consecutive requests of v_p in the queue, each originating from two different tasks. Because $(1/\mu_p) \leq \Delta$ in the M/D/1 queuing system, as indicated in [9], [42], and [43], we have

$$t_p + \frac{1}{\mu_p} \leq t_p + \Delta. \quad (4)$$

Assuming all immediate predecessors of the partition v_p must have been completed before v_i is started, the start times of v_i are $\max_{p \in \text{pre}(i)} t_p$ and $\max_{p \in \text{pre}(i)} \{t_p + \Delta\}$ for the two consecutive tasks. Then, from (4), we obtain

$$\max_{p \in \text{pre}(i)} \left\{ t_p + \frac{1}{\mu_p} \right\} \leq \max_{p \in \text{pre}(i)} \{t_p + \Delta\} \quad (5)$$

which can be rewritten as

$$\max_{p \in \text{pre}(i)} \{t_p + \Delta\} - \max_{p \in \text{pre}(i)} t_p \geq \min_{p \in \text{pre}(i)} \frac{1}{\mu_p}. \quad (6)$$

In (6), the left-hand side of the inequality indicates the interstart time of v_i for two consecutive tasks. Thus, the queue is empty if

$$\max_{p \in \text{pre}(i)} \{t_p + \Delta\} - \max_{p \in \text{pre}(i)} t_p \geq \frac{1}{\mu_i}. \quad (7)$$

That is, if the service time of v_i belonging to the first task $(1/\mu_i)$ is less than the interstart time, then $T_i^{(w)}$ becomes 0.

We denote $\pi(\rho, K)$ as the probability of a full buffer with load ρ in an M/D/1/K queuing system

$$\pi(\rho, K) = 1 - \frac{b_{K-1}}{1 + \rho b_{K-1}}. \quad (8)$$

Given $\pi(\rho, K)$, the probability P_i that partition v_i will be successfully processed is

$$P_i = \begin{cases} \prod_{p \in \text{pre}(i)} P_p, & \text{if } \frac{1}{\mu_i} \leq \min_{p \in \text{pre}(i)} \frac{1}{\mu_p} \\ (1 - \pi(\rho_i, K)) \prod_{p \in \text{pre}(i)} P_p, & \text{otherwise} \end{cases} \quad (9)$$

where the probability of success for the first partition is $1 - \pi(\rho_i, K)$.

2) *Transmission Delay*: Let $B_{d,d'}$ be the bandwidth of the link between IoT devices d and d' , where $B_{d,d'} = \infty$ if $d = d'$ [9], [11]. When $y_p = d$ and $y_i = d'$, the transmission delay between partitions v_p and v_i is as follows:

$$T_{p,i}^{(tr)} = \frac{S_i^{(out)}}{B_{d,d'}} \bigg|_{y_p=d, y_i=d'}. \quad (10)$$

We compute the completion time $T_i^{(cp)}$ of partition v_i as the maximum completion time of its predecessors $p \in \text{pre}(i)$, including the data transmission delay between v_p and v_i [44], [45]

$$T_i^{(cp)} = T_i^{(pr)} + \max_{p \in \text{pre}(i)} \{T_p^{(cp)} + T_{p,i}^{(tr)}\}. \quad (11)$$

If the deployment decisions for the first and last partitions $v_{\hat{n}}, v_{\check{n}}$ are $y_{\hat{n}} = \hat{d}$ and $y_{\check{n}} = \check{d}$, respectively, the total delay is as follows:

$$T_n = \frac{S_{\hat{n}}^{(in)} + S_{\hat{n}}^{(out)}}{B_{d,1}} + \frac{S_{\check{n}}^{(in)}}{B_{1,\hat{d}}} + T_{\check{n}}^{(cp)} + \frac{S_{\check{n}}^{(out)}}{B_{\check{d},1}}. \quad (12)$$

The total delay can be broken down into the sum of four components in (12). The first component is the transmission delay of the input data from the device requesting the DNN model to the gateway and then the output data from the gateway back to the requesting device. The second component is the transmission delay from the gateway to the device, with the input data of the next partition. The third component is the total processing delay and transmission delay between the deployed devices from (11). The last component is the transmission time of the output data of the last partition to the gateway.

C. Energy Consumption

Using u_d , we denote the computing resource utilization of an IoT device d as follows:

$$u_d = \frac{\sum_{i=1}^{|V|} x_i I\{y_i = d\}}{C_d} \quad (13)$$

where $V = \{v_i | 1 \leq n \leq N, v_i \in V_n\}$, C_d is the available computing capacity of device d , and $I\{\cdot\}$ is an indicator function: $I\{\cdot\} = 1$ if condition $\cdot$ holds and 0 otherwise. The energy consumed for computation in the IoT device d is expressed as the summation of energy consumed during idle ($E_d^{(idle)}$) and maximum ($E_d^{(max)}$) utilization

$$E_d^{(cp)} = E_d^{(idle)} + (E_d^{(max)} - E_d^{(idle)})u_d. \quad (14)$$

The energy consumption of IoT devices increases with increasing resource utilization.

Once partition v_i is completed, the output data are transferred to the device with the succeeding partition v_s . The energy consumed from the transmission of v_i 's output data is

$$E_{i,d}^{(tr)} = \sum_{s \in \text{suc}(i)} T_{i,s}^{(tr)} W_d I\{y_i = d\} \quad (15)$$

where W_d is the transmit power of device d . We can then obtain the average energy consumption of device d due to the transmission per λ_i as follows:

$$E_d^{(tr)} = \sum_{i=1}^{|V|} \lambda_i P_i E_{i,d}^{(tr)}. \quad (16)$$

Using (14) and (16), the total energy consumption of device d can be written as

$$E_d = E_d^{(cp)} + E_d^{(tr)}. \quad (17)$$

D. Utility and Optimization

Let (X, Y) denote the partition deployment and resource allocation strategies: $X = \{x_i | 1 \leq i \leq |V|\}$ and $Y = \{y_i | 1 \leq i \leq |V|\}$. Given the delay constraint $\hat{T}_n$ for DNN model n and the energy level $\hat{E}_d$ of the battery at the IoT device d , we model the utility of each type- n DNN model with a sigmoid function, as in [46] and [47]

$$U_n(X, Y) = w P_n \frac{1}{1 + e^{-\beta(\hat{T}_n - T_n)}} + (1 - w) \frac{1}{|D|} \sum_{d \in D} \left(1 - \frac{E_d}{\hat{E}_d}\right) \quad (18)$$

where β is a constant that determines the steepness of the utility curve. As shown in (18), the utility for each model comprises two components: 1) E2E delay for the successfully served request and 2) the average remaining battery capacity per device. The weight w is used to adjust these two factors.

To maximize total utility, we formulate the optimization problem as follows:

$$\arg \max_{X, Y} \sum_{n=1}^N U_n(X, Y) \quad (19)$$

subject to

$$u_d \leq 1 \quad (20)$$

$$\sum_{i=1}^{|V|} S_i^{(M)} I\{y_i = d\} \leq M_d \quad (21)$$

$$E_d(X, Y) \leq \hat{E}_d. \quad (22)$$

For all IoT devices, M_d is the available memory capacity of device d . The constraint in (22) indicates that energy consumption cannot exceed that of the battery.

V. ALGORITHM FOR SOLVING PROBLEM FORMULATION

In this section, we introduce our PRA algorithm. As the problem formulated in the previous section jointly optimizes both partition deployment and resource allocation, the solution becomes highly complex. As in [9], our algorithm is designed to solve the problem formulation in a hierarchical structure: it makes a partition placement decision first and then decides the partition's resource allocation. Noting that the IoT edge environment (i.e., the available computing and communication resources, battery level, and the number of DNN application requests) is complex and dynamic, we develop our resource allocation algorithm using DRL to maximize the reward while

interacting with an environment in real time. Details of the proposed algorithm are presented in the following sections.

A. Partition Deployment

We propose an algorithm that determines the partition deployment Y such that the total cost of the E2E delay and energy consumption (observed in the Setup phase) is minimized. Because x_i is determined later by the DRL-based resource allocation algorithm, we calculate the cost based on the minimum processing delay and energy consumption. We denote the number of containerized partitions deployed in device d under deployment Y as $f(Y, d) = \sum_{y_i \in Y} I(y_i = d)$. Given deployment Y , where $y_i = d$ for partition v_i , assuming an average computing allocation of $x_i = (C_d/f(Y, d))$ for all v_i in d , the processing delay is as follows:

$$T_i^{(c)}(Y) = \frac{S_i^{(\text{in})} \delta_{i,d} f(Y, d)}{C_d}. \quad (23)$$

Because v_i starts once all the predecessors arrive, we take the maximum transmission delay of the output transmission from v_i 's predecessors

$$T_i^{(t)}(Y) = \max_{p \in \text{pre}(i)} T_{p,i}^{(tr)}. \quad (24)$$

Based on the energy consumed for the computation in (13) and (14), we define the additional cost of energy deploying v_i in proportion to the current battery level $\hat{E}_d$ as

$$E_i^{(c)}(Y) = \frac{E_d^{(\text{max})} - E_d^{(\text{idle})}}{\hat{E}_d} \frac{S_i^{(\text{in})} \delta_{i,d}}{C_d}. \quad (25)$$

By using the processing time (23) in proportion to the delay constraint $\hat{T}_n$, transmission time (24), and energy consumption (25) required to satisfy the delay constraint, we define the deployment cost of Y as

$$c(Y) = \sum_{y_i \in Y} \frac{\alpha_c T_i^{(c)}(Y) + \alpha_t T_i^{(t)}(Y) + \alpha_e E_i^{(c)}(Y)}{\hat{T}_n} \quad (26)$$

where α_c , α_t , and α_e are the weights of the processing time, transmission time, and energy consumption ($\alpha_c + \alpha_t + \alpha_e = 1$), respectively.

We take a dynamic programming (DP) approach to find the optimal deployment Y that minimizes cost $c(Y)$ in (26). Partitions are deployed according to their topological order [40]. We denote the current deployment as Y and a candidate deployment as Y' . Y' holds the next partition v_{i+1} that is not yet deployed by Y , which is placed in a device d' . The change in cost between the current deployment Y and the candidate deployment Y' is

$$\begin{aligned} \Delta c = c(Y') - c(Y) = & \frac{1}{\hat{T}_n} \left\{ \alpha_t T_{i+1}^{(t)}(Y') + \alpha_e E_{i+1}^{(c)}(Y') \right. \\ & \left. + \alpha_c \left(T_{i+1}^{(c)}(Y') + \frac{\sum_{y_i \in Y} T_i^{(c)}(Y) I(y_i = d')}{f(Y, d')} \right) \right\}. \end{aligned} \quad (27)$$

Algorithm 1 DP-Based Partition Deployment Algorithm

Input: $Y = \emptyset$

Output: The optimal deployment Y with the minimum cost, $c(Y)$

Initialize: $i = 1$; $CO = [0, 0, \dots]$

```

1: Function Min_Y( $i, CO$ )
2: for  $j = 0$ ;  $j < i$ ;  $j++$  do
3:   for  $d' \in D$  do
4:     for  $p \in \text{pre}(i)$  do
5:       for  $d \in D$  do
6:         Find set  $\mathbb{Y}$  from  $CO$  s.t.  $f(Y, d') = j$ 
7:         if  $\text{pre}(i) \neq \emptyset$  then
8:           Remove  $Y$  from  $\mathbb{Y}$  s.t.  $y_p \neq d$ 
9:         end if
10:        Find  $Y$  having the min  $c(Y)$  from  $\mathbb{Y}$ 
11:        Generate a candidate deployment  $Y'$  for  $Y$  by
        placing  $v_i$  in  $d'$ 
12:        Compute  $\Delta c$  using Eq. (27) and  $c(Y') = c(Y) + \Delta c$ 
        for the found  $Y$ 
13:        Store  $c(Y')$  in  $CO$  for the found  $Y$ 
14:      end for
15:    end for
16:  end for
17: end for
18: if  $i = |V|$  then
19:   return  $Y$  with the min  $c(Y)$  from  $CO$ 
20: else
21:   Min_Y( $i + 1, CO$ )
22: end if

```

Let $\mathbb{Y}_i^{(p,d)}$ be the set of possible current deployments having predecessor p , where Y has $y_p = d$. We find candidate deployment Y' with the minimum cost by additionally deploying y_{i+1} in Y'

$$\min_{Y' \in \mathbb{Y}_{i+1}^{(p,d)}} c(Y') = \min_{Y \in \mathbb{Y}_i^{(p,d)}} c(Y) + \Delta c \quad \forall d \in D \quad (28)$$

such that $f(Y', d') = j + 1$ and $f(Y, d') = j$ for $\forall Y' \in \mathbb{Y}_{i+1}^{(p,d)}$.

The procedure of our DP-based partition deployment algorithm is given in Algorithm 1. After finding the minimum $c(Y)$ such that $y_p = d$ and $f(Y, d') = j$ for each d' , we compute Δc and $c(Y')$ (lines 6–12) and then store the computed value of $c(Y')$ in CO (line 13) only for Y with the minimum $c(Y)$. Because our partition deployment algorithm takes a DP approach, we do not need to compute Δc and store $c(Y')$ for every candidate deployment Y , leading to a reduction in computation time. Specifically, the computational complexity can be reduced from $O(|D|^{|V|})$ to $O(|D|^2|V|^3)$.

B. DRL-Based Resource Allocation

We formulate the resource allocation problem as a Markov decision process (MDP) and adopt a DRL algorithm to find the optimal solution in a continuous space. As large action dimensions complicate resource allocation, we employ an SAC algorithm [48], a state-of-the-art DRL algorithm based on a

maximum entropy framework that has a wide-ranging exploration ability with multimodal reward to efficiently handle computing resource allocation in a continuous space, $X = \{x_i\}$. At each time step t , the agent observes the environment state $s(t)$ and stochastically performs action $a(t)$ following a policy. State $s(t)$ refers to the computing resources, battery level, and number of service requests in the IoT system. After executing action $a(t)$, the agent receives reward $r(t)$ and the environment transitions to a new state $s(t+1)$. This process is repeated until the agent reaches the terminal state, at which the process restarts. The goal of the DRL agent during this learning process is to maximize the total reward gained during the different time-steps of each episode. The key components of our DRL-based resource allocation algorithm, i.e., the observed state, action, and reward are defined as follows. The state vector is

$$s(t) = \{\hat{E}_d, \lambda_n, X(t) | 1 \leq n \leq N \forall d \in D\} \quad (29)$$

where $X(t) = \{x_i\}$ is the set of resource allocations x_i for each v_i at the time step t . For each partition v_i , we denote the resource quota (in %) allocated to v_i by $a_i(t)$.

For device d , the action space is represented as

$$a(t) = \{a_i(t) | g(t) \leq 1, 1 \leq i \leq |V|\} \quad (30)$$

where

$$g(t) = \sum_{i=1}^{|V|} a_i(t) I(y_i = d). \quad (31)$$

To ensure the condition in (30), a Softmax layer is added as the final layer of the actor network. After executing action $a(t)$, resource allocation x_i is determined by $a_i(t)$ as $x_i(t+1) = C_d a_i(t)$ and the agent obtains a new state $s(t+1)$.

Based on the objective function in (19), the reward function can be obtained as

$$r(t) = \sum_{n=1}^N U_n(X(t+1), Y) - \sum_{n=1}^N U_n(X(t), Y) \quad (32)$$

which is the change in total utility from the previous time step. Based on the components defined above, the PRA agent interconnects with the environment of IoT system considered and learns the resource allocation strategy X .

DRL for determining the resource allocation strategy consists of an actor and critic network, each with $J^{(a)}$ and $J^{(c)}$ layers, respectively. Each actor and critic network has the j th layer's feature map that consists of $H_j^{(a)}$ and $H_j^{(c)}$ neurons, respectively. When the number of training episodes is E and the time per episode is T , the total training complexity for the actor-critic network can be expressed as the following multiple of E and T : $O(ET(\sum_{j=2}^{J^{(a)}-1} (H_{j-1}^{(a)} H_j^{(a)} + H_j^{(a)} H_{j+1}^{(a)}) + \sum_{j=2}^{J^{(c)}-1} (H_{j-1}^{(c)} H_j^{(c)} + H_j^{(c)} H_{j+1}^{(c)})))$.

The inference complexity of the pretrained DRL for deciding the resource allocation strategy is $O(\sum_{j=2}^{J^{(a)}-1} (H_{j-1}^{(a)} H_j^{(a)} + H_j^{(a)} H_{j+1}^{(a)}))$ [49]. However, the inference overhead is not a part of the E2E delay of processing DNN inference queries as DRL-based inference is performed separately in the *PRA Phase* (as mentioned in Section III-B).

TABLE III
DEVICE CONFIGURATIONS

Device	Computing capability (GFLOPS)	Memory (GB)	Idle power (W)	Max. power (W)	Battery (W)
d_1, d_2, d_3	5×10^2	4	0.5	4	10
d_4, d_5	10^3	6	4	30	80

VI. PERFORMANCE EVALUATION

We evaluated the performance of our proposed DRL-based algorithm in terms of average E2E delay, utility, and energy cost. The proposed PRA was implemented using Python 3.7 and PyTorch 1.8.

A. Simulation Setup

In our simulation experiments, we consider five IoT devices and one edge node (e.g., home IoT). Table III shows the configurations of the IoT devices, namely, a smartphone, Jetson, and laptop [9], [23], [32]. As an edge node, we consider an NVIDIA Tesla V100 graphics processing unit (GPU) with the computing power and memory capacity set to 4 tera-floating point operations per second (TFLOPS) and 16 GB, respectively, considering that only a portion of the peak performance is typically available in practical scenarios [9]. We set the wireless transmission rate from the IoT devices and edge node to $(100/|D|)$ Mb/s, and the bandwidth between the edge node and the device requesting the DNN model to 100 Mb/s [9], [35]. We assumed a fixed transmission power of 1 W for all devices.

For our simulation, we utilize current popular DNN models, such as ResNet, AlexNet, NiN, and VGG. For each DNN model with DAG topology, there are branches which are parallelizable. Each branch is divided into partitions. Then, horizontal partitioning based on device memory and computing capacity, which is the most common method, is executed such that a partition consists of consecutive DNN layers [28], [34]. Detailed information about each type of DNN, including the structure, computation intensity, and data transmissions between layers, which were obtained from actual execution and simulation modeling, and our partitioning method based on device memory and computing capacity, can be found in our GitHub project [50]. We set the number of DNN models N to 5 and 7.

For the simulation settings of $N = 5$, we utilized one of each model and two of ResNet. For $N = 7$, we utilized two of each model and one of VGG. Each DNN model is generated following a Poisson process with an average arrival rate λ varying from 5 to 30 per s. Subsequently, λ_n is randomly selected from $\lambda \pm 2$ [9].

For the simulation, the weights ω , α_c , α_t , and α_e are evenly distributed across their related factors. We set the delay constraint ($\hat{T}_n$) to 200 ms.

Two scenarios were considered for our evaluation: Scenario 1, which assume that the devices are utilized until battery depletion, and Scenario 2 with the battery level threshold ($\hat{E}_d$) set to 50%.

To demonstrate the efficiency of the proposed PRA, the following benchmarks are utilized for performance comparison.

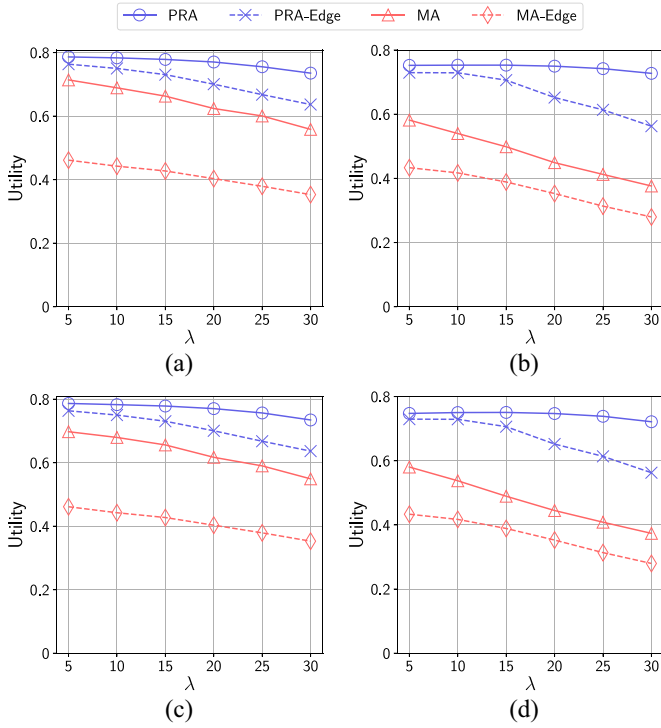


Fig. 2. Utility under different arrival rates with different number of services (left to right) and scenarios (top to bottom). (a) Scenario 1, $N = 5$. (b) Scenario 1, $N = 7$. (c) Scenario 2, $N = 5$. (d) Scenario 2, $N = 7$.

- 1) *Markov Approximation-Based Algorithm (MA)*: We utilize a combination of the greedy placement algorithm and Markov-based resource allocation to find the near optimal utility for our benchmark [7], [9].
- 2) *PRA-Edge*: To test the necessity of the additionally incorporated resources of devices, we compare our algorithm to PRA-Edge, which applies our PRA only to an edge's resources.
- 3) *MA-Edge*: Similar to PRA-Edge, MA is employed considering only the edge's resources.

B. Evaluation Results

First, we evaluate utility in (18), a metric for measuring the service probability, E2E delay, and energy consumption. Results are averaged for all DNN models. As shown in Fig. 2, utility decreases as arrival rate and number of services increase for the three algorithms because more requests cannot be processed with limited resources. Our proposed algorithm maintains the highest utility with all arrival rates for both $N = 5$ and 7 in the two scenarios. Compared with MA, our algorithm shows average improvements of 20.47% and 22.32% (60.01% and 60.61%) for $N = 5$ ($N = 7$) in Scenarios 1 and 2, respectively. We observe from Fig. 2 that both PRA and PRA-Edge achieve higher utility than the MA. This is because they learn to allocate x_i such that utility is maximized, as shown in the reward function in (32). MA-Edge exhibits the lowest utility for both scenarios and arrival rates. We can also notice that utility decreases more rapidly for $N = 7$ than for $N = 5$ because the computing and memory resources (i.e., x_i

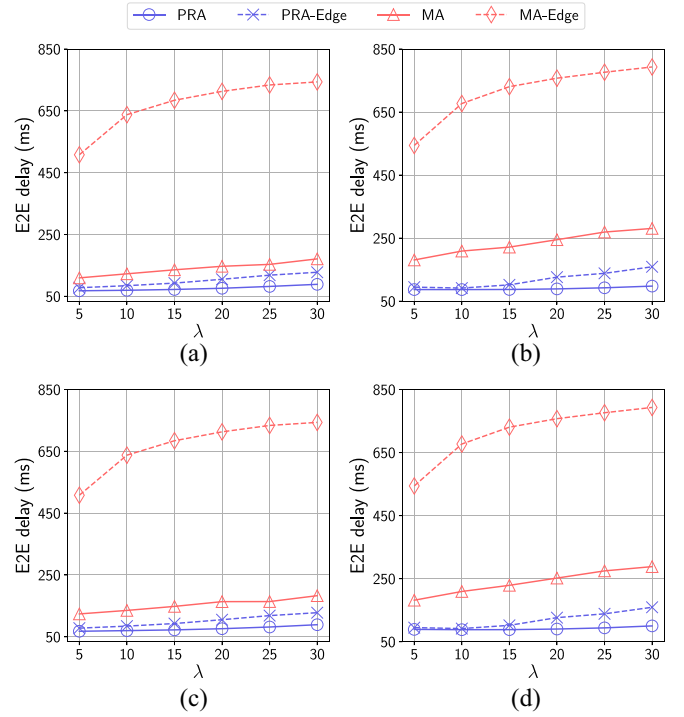


Fig. 3. E2E delay under different arrival rates. (a) Scenario 1, $N = 5$. (b) Scenario 1, $N = 7$. (c) Scenario 2, $N = 5$. (d) Scenario 2, $N = 7$.

and $S_i^{(M)}$) are allocated per container at a greater rate due to the increased partition load.

Fig. 3 shows the average E2E delay. As expected, the E2E delay increases with the arrival rate. It is higher for $N = 7$ than for $N = 5$ because of the increased partition load (i.e., a decrease in x_i and $S_i^{(M)}$) causes the E2E delay to increase, consequently decreasing utility, as shown in Fig. 2. Our proposed algorithm exhibits the lowest E2E delay, displaying the same trend as the utility results. Specifically, it reduces the E2E delay of MA by 45.31% and 50.07% (60.9% and 60.93%) on average for $N = 5$ ($N = 7$) in Scenarios 1 and 2. This is because our algorithm can efficiently utilize more resources, consequently decreasing the waiting time, which contributes to the E2E delay, as indicated by in (3). Compared with PRA-Edge, almost identical improvements of 23.75% and 23.72% (21.47% and 20.19%) can be observed in Scenarios 1 and 2, respectively, for $N = 5$ ($N = 7$). The performance improvement over PRA-Edge results from the difference in resource availability, as PRA-Edge only utilizes the edge server's resources. MA-Edge exhibits the worst performance with the longest waiting time. The trend in Fig. 3 is also shown in Fig. 4, indicating that the waiting delay affects the E2E delay. More specifically, E2E delay is determined by the processing rate and delay that occurs from waiting for the preceding partitions to be completed in (11).

The average waiting delay of DNN partitions for each device is shown in Fig. 4. Our proposed algorithm performed better than both MA and PRA-Edge. Compared to MA, it shows significant performance improvements up to 95.96% and 94.63% for $N = 5$ and $N = 7$ out of the two scenarios. This is because of the same reason that our DRL-based solution is able to maximize utility by learning to reduce E2E delay

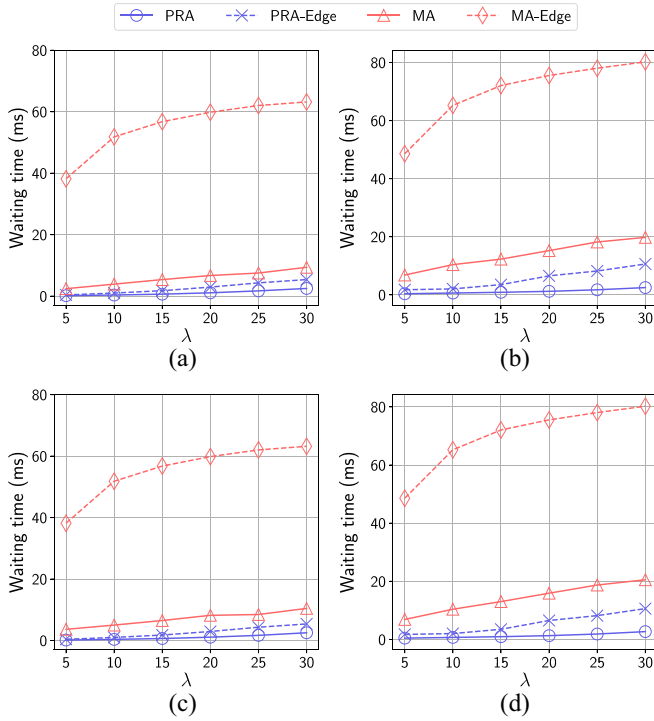


Fig. 4. Waiting time under different arrival rates. (a) Scenario 1, $N = 5$. (b) Scenario 1, $N = 7$. (c) Scenario 2, $N = 5$. (d) Scenario 2, $N = 7$.

TABLE IV
IMPROVEMENT IN ENERGY CONSUMPTION PER DEVICE OVER MA

N	Scenario	d_1	d_2	d_3	d_4	d_5
5	1	21.56%	4.14%	19.31%	44.53%	5.15%
	2	20.81%	4.61%	17.39%	43.00%	5.53%
7	1	21.70%	0.85%	38.77%	38.95%	5.70%
	2	19.94%	0.61%	39.81%	40.23%	6.12%

composing the utility. Compared with PRA-Edge, it achieves a performance improvement of up to 65.2% for $N = 5$ and up to 81.64% for $N = 7$. The greatest performance improvement over PRA-Edge occurs in Scenario 1 owing to the increased resource gap between PRA and PRA-Edge, as PRA can utilize more resources without energy restriction. The proposed algorithm performs better than PRA-Edge because it can utilize IoT device resources to process partitions.

Fig. 5 shows the service probabilities of the three algorithms. Specifically, PRA shows a performance improvement of up to 7.7% and 15.33% (2.72% and 9.76%) compared to MA (PRA-Edge) for $N = 5$ and $N = 7$, respectively. This is due to the reduced waiting delay of the PRA algorithm and its utility model (18), which maximizes the service probability. In other words, a reduced waiting delay enables PRA to accept more partitions, resulting in a higher service probability. As in the utility results, MA-Edge shows the highest E2E delay and the lowest service probability. Table IV shows the improvements in energy consumption of the proposed algorithm over MA for each device at the end of the run. Our algorithm shows improvements over MA in both Scenarios 1 and 2 because it consumes less energy, as our reward accounts for the energy consumption in proportion to the remaining battery, as can be seen in the utility (18), and reward (32).

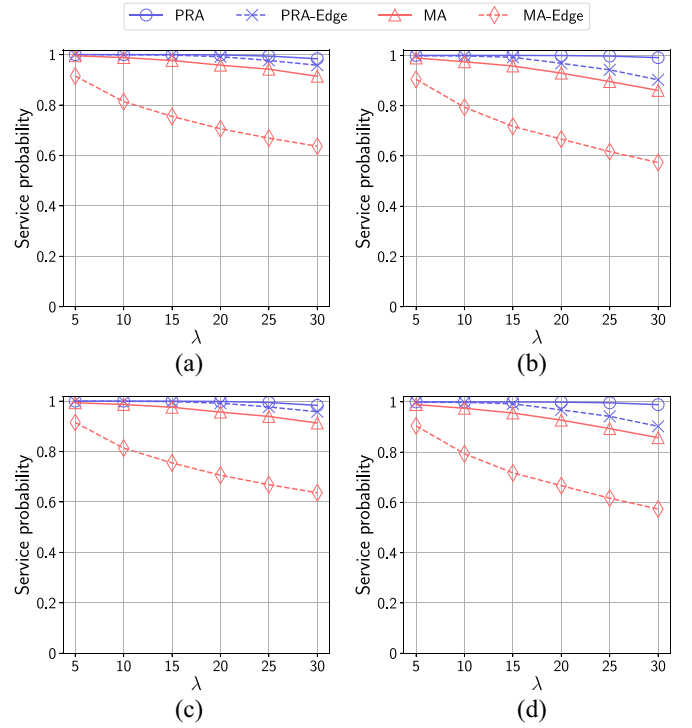


Fig. 5. Service probability under different arrival rates. (a) Scenario 1, $N = 5$. (b) Scenario 1, $N = 7$. (c) Scenario 2, $N = 5$. (d) Scenario 2, $N = 7$.

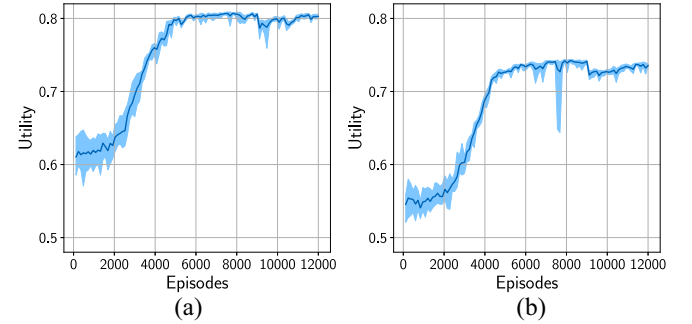


Fig. 6. Training reward versus episodes for PRA. (a) $N = 5$. (b) $N = 7$.

Fig. 6 illustrates the utility versus episode number to depict the learning performance for PRA, where the utility is the average of 80 independent training trials for both $N = 5$ and $N = 7$. The utility increases as the training step increases and converges after 6000 episodes for $N = 5$ and 7.

C. Performance in Dynamic Network Environment

From the original Scenario 1, we consider the addition of devices d_6 and d_7 each with equal capacity to any of d_1 , d_2 , and d_3 as shown in Fig. 7. When $N = 5$, we did not observe an improvement in performance with only the addition of devices as the deployed PRA strategy continued to utilize only five devices, although six or seven devices are available. This was due to the increased transmission delay evening out the decrease in processing delay when utilizing additional computational resources. However, when $N = 7$, the deployed strategy utilized all six or seven devices as the more computational resources are requested, compared to when $N = 5$.

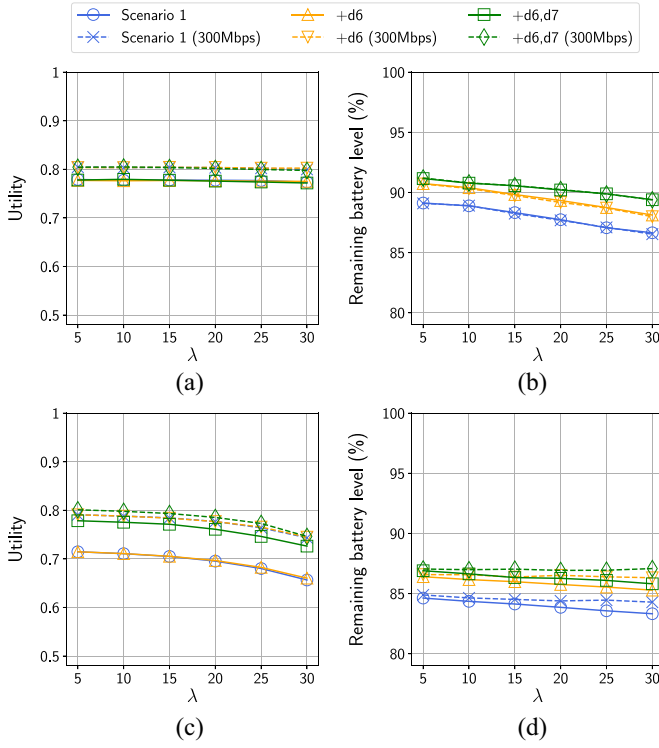


Fig. 7. Utility and energy consumption with additional devices and bandwidth. (a) and (b) $N = 5$. (c) and (d) $N = 7$.

Accordingly, with $N = 7$ and the additional computational resources of just d_6 , the utility increased up to 0.45% compared to the Scenario 1. With the addition of both d_6 and d_7 , the utility further increased up to 9.35%. In the case of energy, average remaining battery level did improve overall with the addition of devices as more energy and computational resource was available.

To observe the effects of bandwidth, we tested Scenario 1 in a network environment with bandwidth increased from $(100/|D|)$ Mb/s to $(300/|D|)$ Mb/s, as shown in Fig. 7. In the case of $N = 5$, we observed an increase of 3% in utility on average for all cases including Scenario 1, addition of d_6 and addition of both d_6 and d_7 . For $N = 7$, average increases of 11% and 3% were observed with the addition of d_6 and addition of both d_6 and d_7 , respectively. This relative increase in utility resulted from the decreased transmission delay as bandwidth was increased. In other words, to fully benefit from all available devices, our scheme requires higher bandwidth. As for the change in energy performance shown in Fig. 7(b), measured in average remaining battery percentage, there was an increase in the remaining battery of up to 1.29%, 1.3%, and 1.47% in the cases of Scenario 1, addition of d_6 and addition of both d_6 and d_7 , respectively. The increase of remaining battery indicated that there was a decrease in the energy consumption.

D. Partition Deployment on Experimental Testbed

In this experiment, we prove real-world feasibility and efficiency of the proposed algorithm. As shown in Fig. 8, the testbed comprises an edge node equipped with a Ryzen 7 CPU,

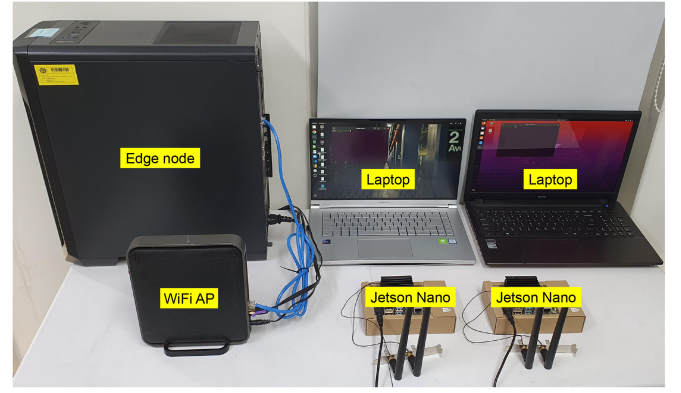


Fig. 8. Our experimental testbed of two laptops, two Jetson nanos, and one edge node.

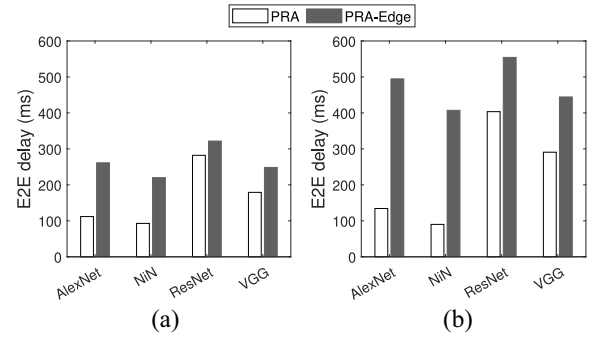


Fig. 9. E2E delay on our heterogeneous testbed when (a) $\lambda = 5$ and (b) $\lambda = 10$.

GTX 1070 GPU, 32 GB RAM, two Jetson Nanos, and two i5 CPU equipped laptops, one with 840M GPU and 4-GB RAM, and the other with MX250 GPU and 8 GB RAM, which are similar to those of [32].

We created a Kubernetes cluster using kubeadm for cluster initialization [32], [39]. For our test models, we utilized ResNet, AlexNet, NiN, and VGG built on Tensorflow packaged with Docker files. We tested our algorithm for $\lambda = 5$ and 10.

In terms of the average E2E delay, shown in Fig. 9, PRA showed a 37% and 52% increase in performance over PRA-Edge when λ is 5 and 10, respectively. This performance enhancement is due to the difference in computing capacity (C_d) between PRA and PRA-Edge: the former has additional resources of the IoT devices. Looking at the increase in performance by model type when $\lambda = 5$, AlexNet, NiN, ResNet, and VGG showed an increase of 57%, 58%, 12%, and 37%, respectively. ResNet had the least performance gain because of its high computation intensity, as a model with high computation intensity requires proportionally high computing resources to show performance improvement similar to that of others.

VII. CONCLUSION

In this study, we proposed an optimal partition PRA algorithm for multiple DNNs in the IoT system. As the number of DNN-based applications to be processed increases, more

computing resources are needed, leading to waiting or blocking at the edge or the IoT device. Thus, the waiting time and service probability for multiple DNN models were analyzed using M/D/1/K queuing. Subsequently, a problem formulation that maximizes the utility consisting of the E2E delay and average remaining battery capacity per device was developed. To address the resource scarcity at the IoT edge, the available resources of nearby IoT devices were incorporated in to the problem formulation. To efficiently solve the problem, the PRA algorithm decides the containerized partition placement and then allocates the resources to the placed partitions, for which a DP approach and DRL are adopted, respectively. The simulation results confirmed the efficiency of the proposed PRA strategy in terms of utility compared to existing solutions. We also assessed our container-based solutions in a real IoT testbed where Kubernetes is used to deploy containerized partitions on heterogeneous devices. The results measured in the testbed demonstrated that PRA improves the delay performance over the approach using only the edge server by utilizing the resources of IoT devices, implying PRA's actual feasibility.

In future work, we plan to develop an incentive model that can motivate IoT devices to voluntarily participate in collaborative computing. By incorporating the developed model, we expect to extend our proposed strategy to be applied to IoT devices in public spaces.

REFERENCES

- [1] Y. Huang, X. Qiao, S. Dustdar, J. Zhang, and J. Li, "Toward decentralized and collaborative deep learning inference for intelligent IoT devices," *IEEE Netw.*, vol. 36, no. 1, pp. 59–68, Jan./Feb. 2022.
- [2] H. Wang et al., "A comprehensive survey on training acceleration for large machine learning models in IoT," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 939–963, Jan. 2022.
- [3] E. Baccour et al., "Pervasive AI for IoT applications: A survey on resource-efficient distributed artificial intelligence," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2366–2418, 4th Quart., 2022.
- [4] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 869–904, 2nd Quart., 2020.
- [5] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, 4th Quart., 2021.
- [6] Z. Chang, L. Liu, X. Guo, and Q. Sheng, "Dynamic resource allocation and computation offloading for IoT fog computing system," *IEEE Trans. Ind. Informat.*, vol. 17, no. 5, pp. 3348–3357, May 2021.
- [7] S. Zhang et al., "Adaptive configuration selection and bandwidth allocation for edge-based video analytics," *IEEE/ACM Trans. Netw.*, vol. 30, no. 1, pp. 285–298, Feb. 2022.
- [8] L. Chen et al., "IoT microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 16, pp. 12610–12622, Aug. 2021.
- [9] W. He, S. Guo, S. Guo, X. Qiu, and F. Qi, "Joint DNN partition deployment and resource allocation for delay-sensitive deep learning inference in IoT," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9241–9254, Oct. 2020.
- [10] L. Zhou, M. H. Samavatian, A. Bacha, S. Majumdar, and R. Teodorescu, "Adaptive parallel execution of deep neural networks on heterogeneous edge devices," in *Proc. 4th ACM/IEEE Symp. Edge Comput. (SEC)*, Nov. 2019, pp. 195–208.
- [11] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
- [12] S. Yi, Z. Hao, Q. Zhang, Q. Zhang, W. Shi, and Q. Li, "LAVEA: Latency-aware video analytics on edge computing platform," in *Proc. ACM/IEEE Symp. Edge Comput. (SEC)*, Oct. 2017, pp. 1–13.
- [13] S. Wang et al., "When edge meets learning: Adaptive control for resource-constrained distributed machine learning," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2018, pp. 63–71.
- [14] R. Morabito, I. Farris, A. Iera, and T. Taleb, "Evaluating performance of containerized IoT services for clustered devices at the network edge," *IEEE Internet Things J.*, vol. 4, no. 4, pp. 1019–1030, Aug. 2017.
- [15] R. Morabito, "Virtualization on Internet of Things edge devices with container technologies: A performance evaluation," *IEEE Access*, vol. 5, pp. 8835–8850, 2017.
- [16] S. Boovaraghavan, A. Maravi, P. Mallela, and Y. Agarwal, "MLIoT: An end-to-end machine learning system for the Internet-of-Things," in *Proc. Int. Conf. Internet Things Des. Implement. (IoTDI)*, May 2021, pp. 169–181.
- [17] Y. Jin et al., "Self-aware distributed deep learning framework for heterogeneous IoT edge devices," *Future Gener. Comput. Syst.*, vol. 125, pp. 908–920, Dec. 2021.
- [18] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2980–2988.
- [19] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 815–823.
- [20] B. Cox, J. Galjaard, A. Ghiassi, R. Birke, and L. Y. Chen, "Masa: Responsive multi-DNN inference on the edge," in *Proc. IEEE Int. Conf. Pervas. Comput. Commun. (PerCom)*, Mar. 2021, pp. 446–447.
- [21] B. Mao, F. Tang, Y. Kawamoto, and N. Kato, "Optimizing computation offloading in satellite-UAV-served 6G IoT: A deep learning approach," *IEEE Netw.*, vol. 35, no. 4, pp. 102–108, Jul./Aug. 2021.
- [22] B. Mao, F. Tang, Y. Kawamoto, and N. Kato, "AI models for green communications towards 6G," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 1, pp. 210–247, 1st Quart., 2022.
- [23] M. Xue, H. Wu, R. Li, M. Xu, and P. Jiao, "EosDNN: An efficient offloading scheme for DNN inference acceleration in local-edge-cloud collaborative environments," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 248–264, Mar. 2022.
- [24] J. Lin, P. Yang, W. Wu, N. Zhang, T. Han, and L. Yu, "Edge learning for low-latency video analytics: Query scheduling and resource allocation," in *Proc. Int. Conf. Mobile Ad Hoc Smart Syst. (MASS)*, Oct. 2021, pp. 252–259.
- [25] Z. Xu et al., "Energy-aware inference offloading for DNN-driven applications in mobile edge clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 4, pp. 799–814, Apr. 2021.
- [26] W. Shi, Y. Hou, S. Zhou, Z. Niu, Y. Zhang, and L. Geng, "Improving device-edge cooperative inference of deep learning via 2-step pruning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr./May 2019, pp. 1–6.
- [27] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 447–457, Jan. 2020.
- [28] X. Chen, J. Zhang, B. Lin, Z. Chen, K. Wolter, and G. Min, "Energy-efficient offloading for DNN-based smart IoT systems in cloud-edge environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 683–697, Mar. 2022.
- [29] L. Zeng, E. Li, Z. Zhou, and X. Chen, "Boomerang: On-demand cooperative deep neural network inference for edge intelligence on the industrial Internet of Things," *IEEE Netw.*, vol. 33, no. 5, pp. 96–103, Sep./Oct. 2019.
- [30] X. Tang, X. Chen, L. Zeng, S. Yu, and L. Chen, "Joint multiuser DNN partitioning and computational resource allocation for collaborative edge intelligence," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9511–9522, Jun. 2021.
- [31] H. Sami and A. Mourad, "Towards dynamic on-demand fog computing formation based on containerization technology," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, Dec. 2018, pp. 960–965.
- [32] H. Sami and A. Mourad, "Dynamic on-demand fog formation offering on-the-fly IoT service deployment," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 1026–1039, Jun. 2020.
- [33] H. Sami, A. Mourad, and W. El-Hajj, "Vehicular-OBUs-as-on-demand-fogs: Resource and context aware deployment of containerized micro-services," *IEEE/ACM Trans. Netw.*, vol. 28, no. 2, pp. 778–790, Apr. 2020.

- [34] Z. Chang, S. Liu, X. Xiong, Z. Cai, and G. Tu, "A survey of recent advances in edge-computing-powered artificial intelligence of things," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13849–13875, Sep. 2021.
- [35] H. Wu, Z. Zhang, C. Guan, K. Wolter, and M. Xu, "Collaborate edge and cloud computing with distributed deep learning for smart city Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8099–8110, Sep. 2020.
- [36] J. Wang, L. Zhao, J. Liu, and N. Kato, "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1529–1541, Jul./Sep. 2021.
- [37] "Home edge project—Home edge—LF edge confluence." 2021. [Online]. Available: <https://www.lfedge.org/projects/homeedge>
- [38] Z. Xu, W. Gong, Q. Xia, W. Liang, O. F. Rana, and G. Wu, "NFV-enabled IoT service provisioning in mobile edge clouds," *IEEE Trans. Mobile Comput.*, vol. 20, no. 5, pp. 1892–1906, May 2021.
- [39] "Kubernetes documentation—Kubernetes." 2022. [Online]. Available: <https://kubernetes.io/docs/home>
- [40] L. Zeng and M. Zhao, "Improved dynamic programming for the shortest path problem with resource constraints in DAG," in *Proc. IEEE 4th Int. Conf. Comput. Commun. (ICCC)*, Dec. 2018, pp. 2278–2282.
- [41] O. Brun and J.-M. Garcia, "Analytical solution of finite capacity M/D/1 queues," *J. Appl. Probab.*, vol. 37, no. 4, pp. 1092–1098, Dec. 2000.
- [42] Q. Ye, J. Li, K. Qu, W. Zhuang, X. S. Shen, and X. Li, "End-to-end quality of service in 5G networks: Examining the effectiveness of a network slicing framework," *IEEE Veh. Technol. Mag.*, vol. 13, no. 2, pp. 65–74, Jun. 2018.
- [43] Q. Ye, W. Zhuang, X. Li, and J. Rao, "End-to-end delay modeling for embedded VNF chains in 5G core networks," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 692–704, Feb. 2019.
- [44] D. Kimovski, N. Mehran, C. E. Kerth, and R. Prodan, "Mobility-aware IoT application placement in the cloud—Edge continuum," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3358–3371, Nov./Dec. 2022.
- [45] Y. Duan and J. Wu, "Computation offloading scheduling for deep neural network inference in mobile computing," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Serv. (IWQoS)*, Jun. 2021, pp. 1–10.
- [46] M. Sheng, C. Xu, X. Wang, Y. Zhang, W. Han, and J. Li, "Utility-based resource allocation for multi-channel decentralized networks," *IEEE Trans. Commun.*, vol. 62, no. 10, pp. 3610–3620, Oct. 2014.
- [47] G. Sun, K. Xiong, G. O. Boateng, D. Ayepah-Mensah, G. Liu, and W. Jiang, "Autonomous resource provisioning and resource customization for mixed traffics in virtualized radio access network," *IEEE Syst. J.*, vol. 13, no. 3, pp. 2454–2465, Sep. 2019.
- [48] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 80, Jul. 2018, pp. 1861–1870.
- [49] H. Yang, J. Zhao, Z. Xiong, K.-Y. Lam, S. Sun, and L. Xiao, "Privacy-preserving federated learning for UAV-enabled networks: Learning-based joint scheduling and resource management," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3144–3159, Oct. 2021.
- [50] "PRA simulation configuration." 2022. [Online]. Available: https://github.com/winet-PRA/sim_config